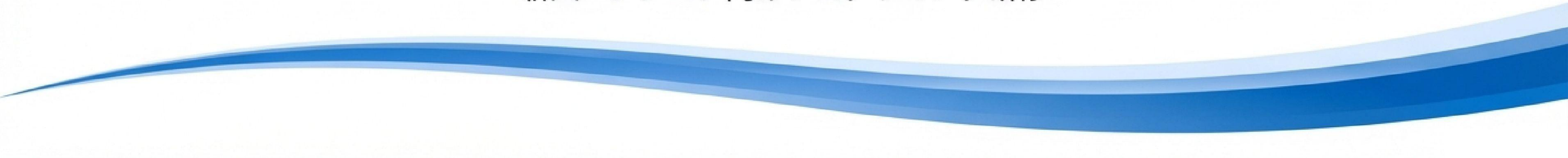


Day1 : Spring Bootによる Webアプリ開発の基礎

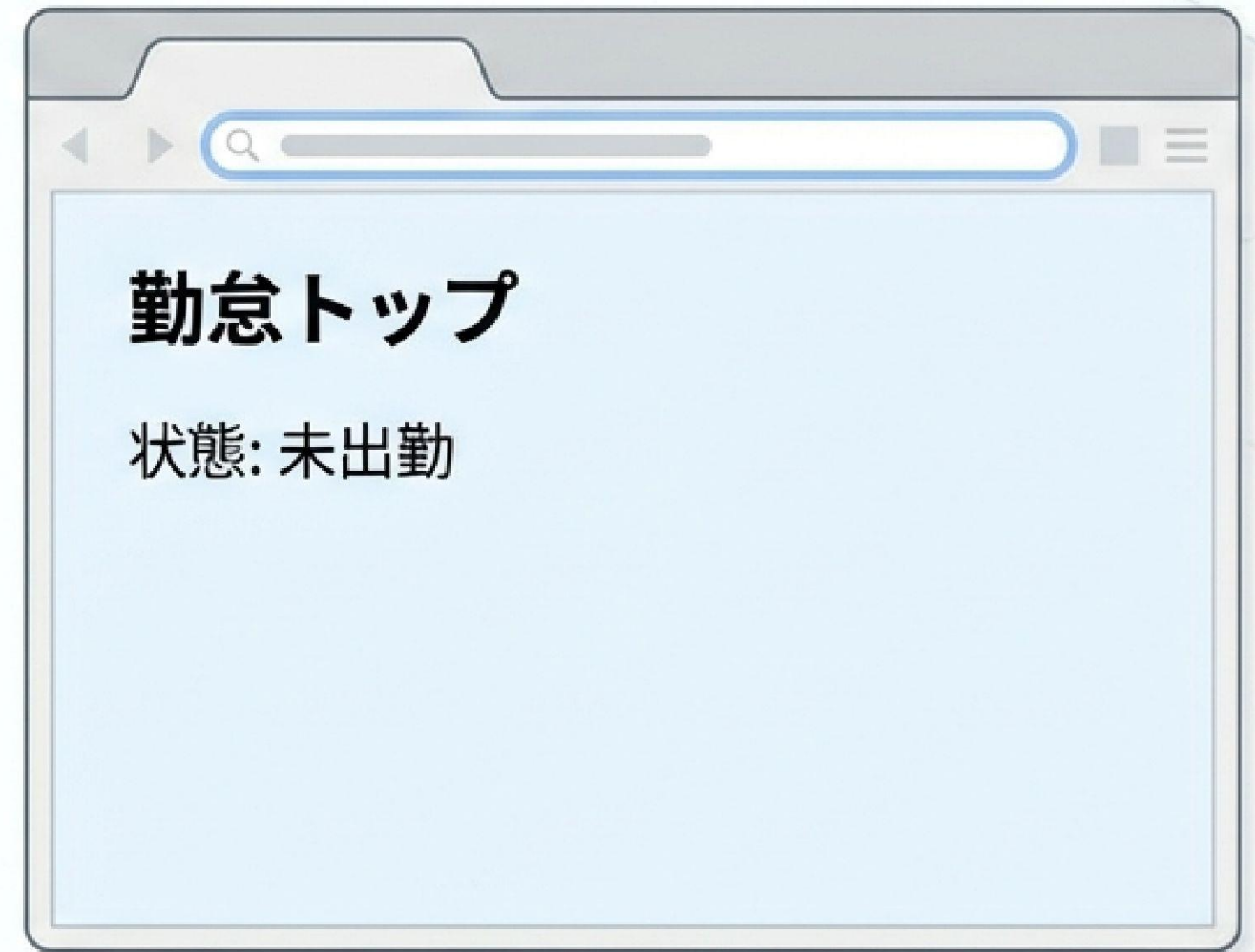
最小構成のMVC (ControllerとThymeleaf)

新人エンジニア向けプログラミング研修



本日のゴール

- ✓ 1. Spring Bootアプリを「ゼロから」起動できる。
- ✓ 2. MVC (Controller → Template) の仕組みを理解する。
- ✓ 3. ブラウザに「勤怠トップ画面」を表示する。



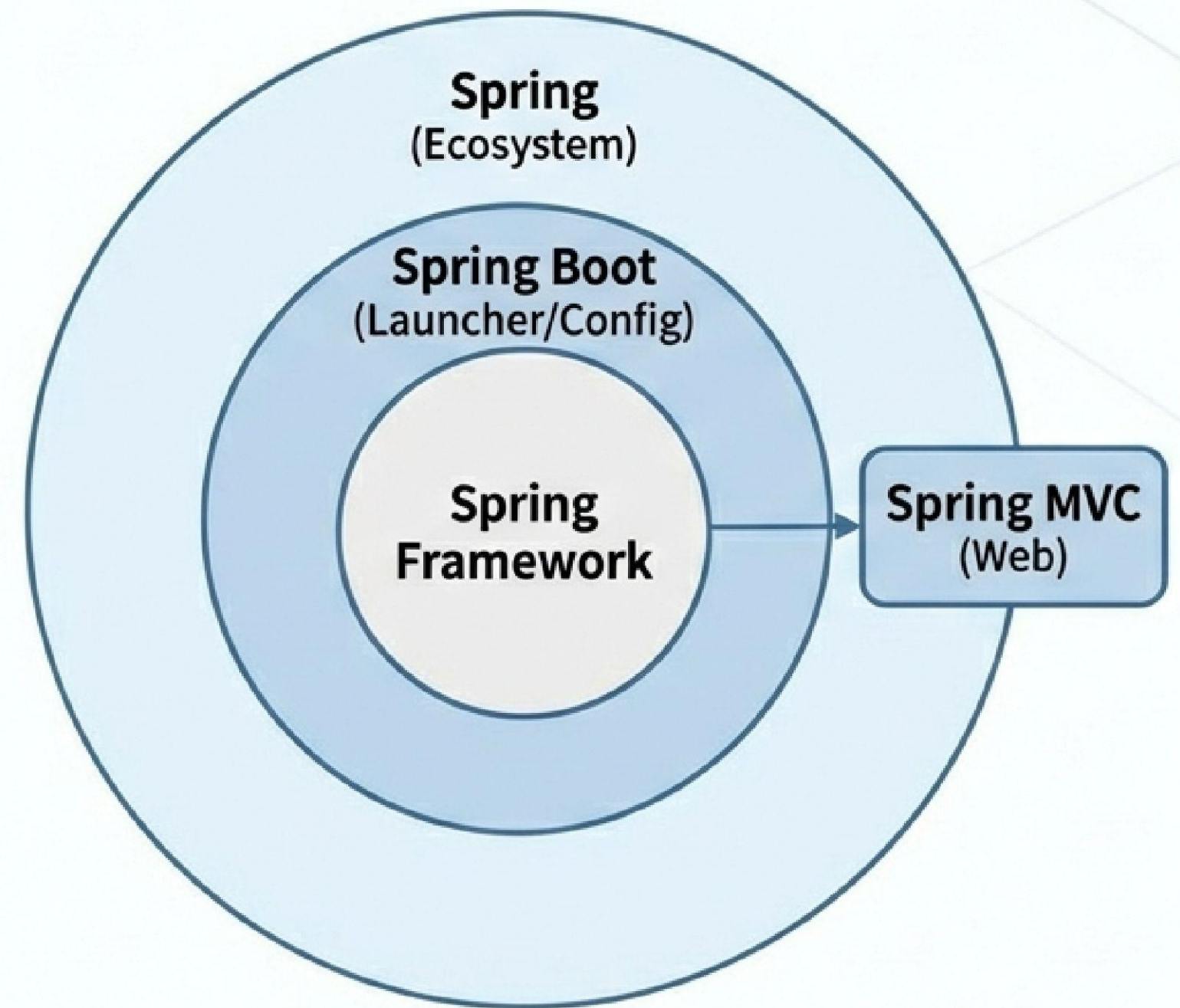
最終的なアウトプットイメージ

Spring Bootとは（全体像）

Spring Framework：Java開発の基盤（機能の集合体）。

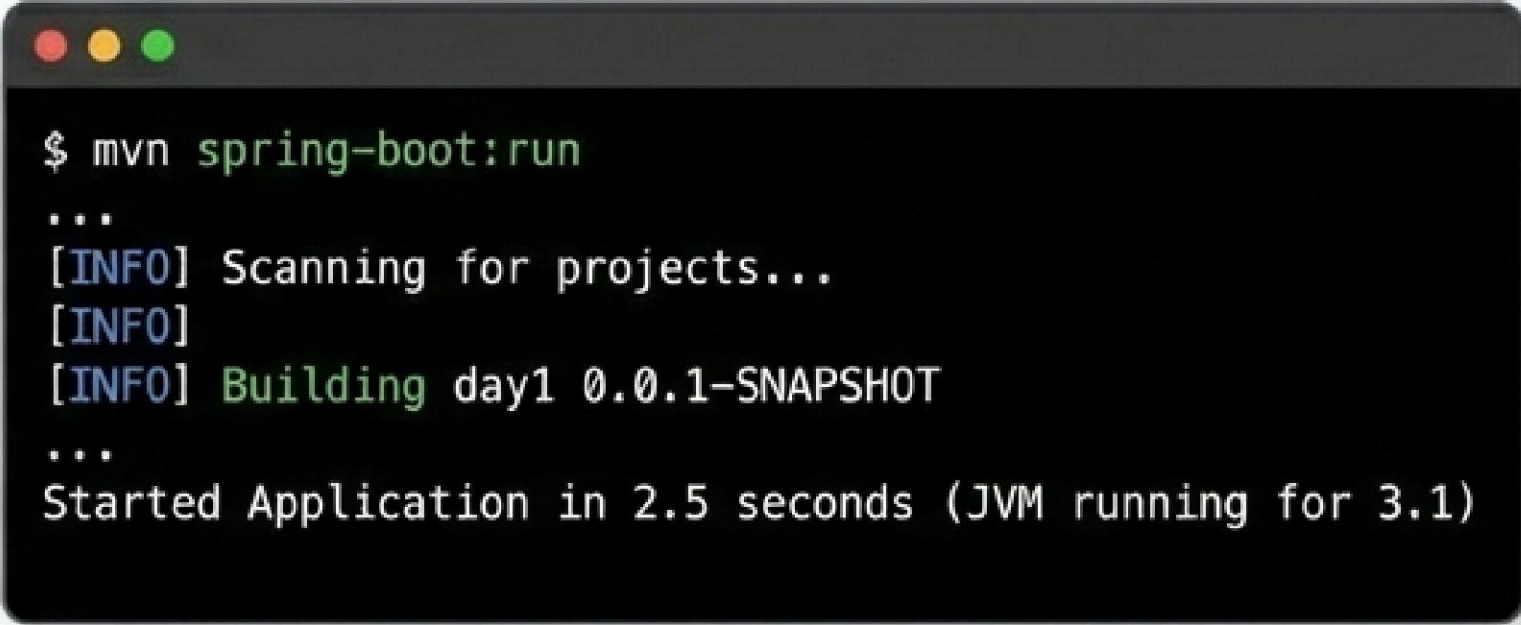
Spring Boot：面倒な設定を自動化し、すぐに起動できる仕組み。

Spring MVC：Web画面やHTTP通信を扱う機能。



Maven超入門

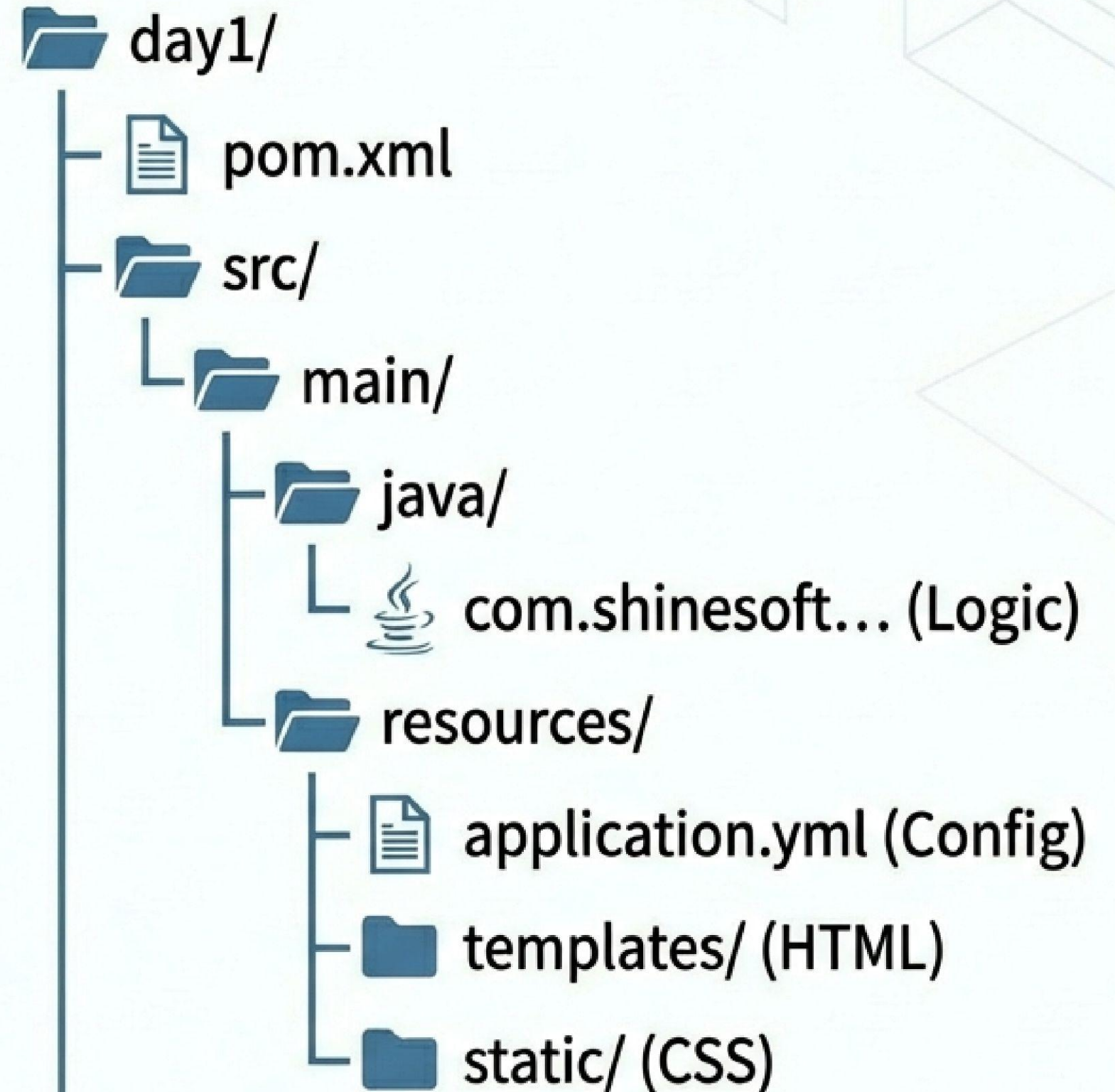
- 役割：ライブラリ管理とビルドの自動化。
- **pom.xml**：「何を使うか（依存関係）」を書く設計図。
- **重要コマンド**：
 - `mvn clean` (掃除)
 - `mvn spring-boot:run` (起動)

A terminal window with a dark background and light-colored text. It shows the execution of the Maven command 'mvn spring-boot:run'. The output includes status messages like '[INFO] Scanning for projects...' and '[INFO] Building day1 0.0.1-SNAPSHOT', followed by a success message 'Started Application in 2.5 seconds (JVM running for 3.1)'.

```
$ mvn spring-boot:run
...
[INFO] Scanning for projects...
[INFO]
[INFO] Building day1 0.0.1-SNAPSHOT
...
Started Application in 2.5 seconds (JVM running for 3.1)
```


ディレクトリ構成と設定

- **src/main/java** : Javaコード (Controllerなど) を置く。
- **src/main/resources** : 設定ファイルやHTMLを置く。
- **pom.xml** : プロジェクト直下に配置。



アプリケーションの起動起点

- **@SpringBootApplication** : 「これがSpring Bootアプリだ」という宣言。
- **mainメソッド** : Javaプログラムの開始地点。
- **SpringApplication.run** : アプリ全体を立ち上げる命令。

```
AttendanceManagementApplication.java

@SpringBootApplication
public class AttendanceManagementApplication {

    public static void main(String[] args) {
        // ここからアプリが起動する
        SpringApplication.run(
            AttendanceManagementApplication.class, args
        );
    }
}
```


Controller（リクエストの受け口）

- **@Controller**：画面遷移を制御するクラス。
- **@GetMapping("/")**：URL「/」へのアクセスを処理。
- **model.addAttribute**：画面に渡したいデータを詰める。

```
HomeController.java

@Controller
public class HomeController {

    @GetMapping("/")
    public String index(Model model) {
        // 画面に渡すデータ
        model.addAttribute("statusLabel", "未出勤");

        // 表示するHTMLファイル名 (index.html)
        return "index";
    }
}
```

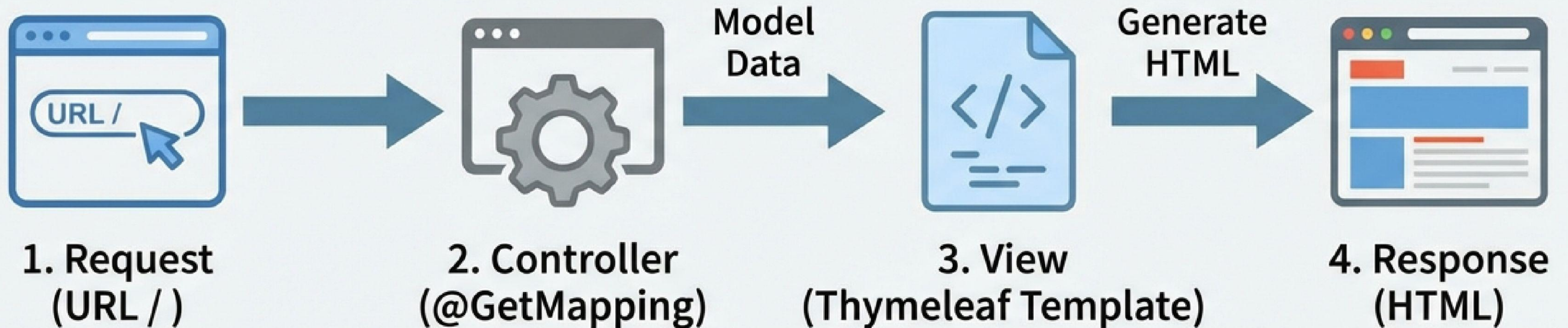

Template（画面の表示）

- Thymeleaf：HTMLにJavaのデータを埋め込むエンジン。
- 配置場所：
src/main/resources/
templates
- th:text：Controllerから渡された値を表示する属性。



```
index.html
<html xmlns:th="http://www.thymeleaf.org">
  <body>
    <h1>勤怠トップ</h1>
    <!-- ControllerのstatusLabelを表示 -->
    <p th:text="${statusLabel}">状態</p>
  </body>
</html>
```


MVCの連携フロー（全体の流れ）



データ（Model）がテンプレートに埋め込まれ、HTMLとしてブラウザに返される

CSSによる装飾

- **src/main/resources/static:** CSSや画像の置き場所。
- Thymeleafでの読み込み
み`th:href`を使用する。

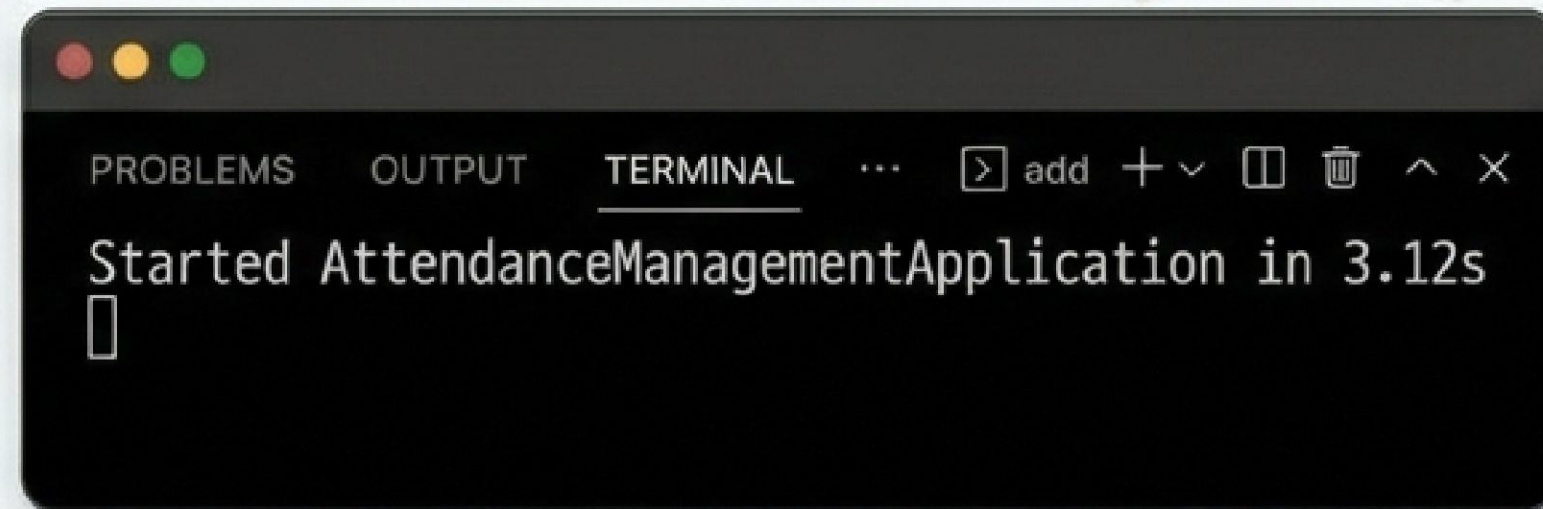
```
/* static/styles.css */
:root {
    --bg: #dbeafe; /* 背景色変数 */
}

body {
    background: var(--bg);
}
```

```
<!-- テンプレートHTMLでの読み込み -->
<link th:href="@{/styles.css}" rel="stylesheet">
```


演習：起動と確認

1. VS Codeのターミナルを開く。
2. ``mvn spring-boot:run``を実行。
3. ログ (Started...) を確認。
4. ブラウザで
``http://localhost:8080``へアクセス。



まとめ

本日の成果

- Spring Bootの最小構成（App + Controller + Template）を作成。
- ブラウザからのリクエストが処理され、画面が表示されるMVCの流れを理解。

Day2（次回）の予告

- 現在は「表示するだけ」の状態。
- 次回は、「出勤」「退勤」ボタンを押し、Javaでロジックを処理する機能を実装します。